Travail noté 2 - INF 1220

Nom : *Lépine*
Prénom : *Alexandra*
Code permanent : *LEPA18549607*
Date de remise : 8 mars 2026

Question #1

Analyse préliminaire

Il faut créer une classe et ses méthodes permettant de calculer le périmètre d'un cercle et l'aire de la surface délimitée par le cerle. La classe doit se nommer Cercle Il faut avoir 3 méthodes :

- 1 constructeur recevant un rayon
- la méthode de calcul de l'air
- la méthode de calcul du périmètre

À partir de Java, je vais donc créer une classe ainsi que ses méthodes permettant de calculer le périmèetre et l'aire d'un cercle, en incluant les 3 méthodes demandées. Dans mon code, je dois tenir compte qu'il n'est pas possible d'utiliser un rayon de zéro ou un rayon négatif et documenter la réaction du code.

Suite à l'indice, dans claude.ia, j'ai saisie l'énoncé : Rendez-vous sur le site de votre intelligence artificielle préférée (Grok, ChatGPT, Gemini, etc.) et saisissez l'énoncé dans la boîte de saisie: « Veuillez créer une classe et ses méthodes permettant de calculer le périmètre d'un cercle et l'aire de la surface délimitée par le cercle. Vous devez avoir 3 méthodes : 1 constructeur recevant un rayon, la méthode de calcul de l'aire et la méthode de calcul du périmètre. La classe doit se nommer Cercle. »

Dans la version de Claude.ia (Sonnet 4.6), du 7 mars 2026, j'ai obtenu le résultat suivant :

> Voici la classe Cercle dans plusieurs langages populaires :

> Python pythonimport math

class Cercle: def **init**(self, rayon): self.rayon = rayon

```
def aire(self):
    return math.pi * self.rayon ** 2

def perimetre(self):
    return 2 * math.pi * self.rayon
```

> ☕ Java javapublic class Cercle { private double rayon;

```
public Cercle(double rayon) {
    this.rayon = rayon;
}
```

```
    public double aire() {
        return Math.PI * rayon * rayon;
    }

    public double perimetre() {
        return 2 * Math.PI * rayon;
    }
```

}

> 💻 C# csharpusing System;

public class Cercle { private double rayon;

```
    public Cercle(double rayon) {
        this.rayon = rayon;
    }

    public double Aire() {
        return Math.PI * rayon * rayon;
    }

    public double Perimetre() {
        return 2 * Math.PI * rayon;
    }
```

}

> 🌐 JavaScript javascriptclass Cercle { constructor(rayon) { this.rayon = rayon; }

```
    aire() {
        return Math.PI * this.rayon ** 2;
    }

    perimetre() {
        return 2 * Math.PI * this.rayon;
    }
```

}

Code source

Code Cercle

```java
public class Cercle {
    private double rayon;

    public Cercle (double rayon) {
        if (rayon < 0 ) {
            rayon = 0;
        }
        this.rayon = rayon;
    }
    public double aire() {
        return Math.PI * rayon * rayon;
        }
        public double perimetre () {
            return 2 * Math.PI * rayon ;
        }
    }
```

Description de la classe

La classe est déclaré public pour qu'elle soit accessible de n'importe quel autre fichier ou classe d'un projet. Cette visibilité est essentiel lorsqu'une classe est destionée à être utilisé par d'autres programmeurs.

L'attribut private double rayon. Pour débuter, l'attribut private permet d'encapsculé le rayon à l'intérieur de la classe, ce qui fait en sorte qu'un autre programmeur ne peut pas directement modifier le rayon suite à la création de l'objet, ceci protège l'intégrité des données. Par la suite, nous utilisons double plutôt que int pour permettre des rayons ayant des valeurs décimales.

Le constructeur est public Cercle (double rayon). Le constructeur est public pour qu'on puisse créer des cercles new Cercle(). Le constructeur reçoit le rayon en paramèetre et l'assigne à this.rayon. This distingue l'attribut de la classe du paramètre local.

Dans le cas où le rayon est nul (0), le code va produire une aire de 0.0 et un périmètre de 0.0.

Dans le cas où le rayon est négatif, le code va ramené automatiquement le rayon à 0 plutôt que de planter le programme ou retourner des résultats absude. Un rayon négatif n'a pas de sens géométrique.

La méthode public double aire(). Nous avons utilisé public étant donné que la méthode est accessible depuis l'extérieur de la classe. La formule de l'aire d'un cercle est $\pi * r^2$, pour appliquer cette formule, nous utilisons Math.PI et retourne un double pour que le résultat de l'aire puisse contenir des décimales.

La méthode public aire(). Nous avons utilisé la même logique que pour la méthode public double aire (). Toutefois, la formule utilisé pour calculé le périmèetre du cercle est $2\pi r$ et comme pour l'aire nous avons utilisé retourne un double pour que le résultat du périmètre puisse contenir des décimales.

Question 2

Analyse préliminaire

Un programmeur souhaite représenter la valeur 10.000000000000001 en Java. Celui-ci a écrit le programme relié à la question 2. Il faut exécutez le programme et expliquez le résultat obtenu.

code source

Code question 2

```
class Main {
  public static void main(String[] args) {
   double x =  10.000000000000001;
   System.out.println(x);
  }
}
```

Sortie du programme

```
10.000000000000002
```

Description du programme

Le code du programmeur utilise le type double qui est la norme IEEE754 sur 64 bits en Java. Nous obtenons comme résultat 10.000000000000002 étant donné que 10.000000000000001 ne peut être représentée en binaire IEEE754. Le nombre flottant le plus proche est 10.000000000000002.

Question 3

Analyse préliminaire

Il faut déterminer la valeur de 'a' aprèes les lignes de code et expliquer pourquoi nous obtenons ce résultat. Les lignes code que nous avons sont les suivants : int i = 3; int a = i++;

Description du programme.

La valeur de i = 3 Dans la ligne de code int a = i++, i++ est post-incrémental, donc nous assignons la valeur 3 à 'a' puis par la suite la valeur de 'a' n'est pas modifié par la post-incrémentation. La valeur de 'a' sera donc de 3.

Question 4

Analyse préliminaire

À partir du programme de la question #4, il faut déterminé ce qui sera affiché par la ligne System.out.println(chaine) et expliquer pourquoi nous avons ce résultat en donnant une réponse concise.

Code source

TestMethode

```java
public class TestMethode {

    public static void test(String test)
    {
        test = test+test;
    }

    public static void main(String[] args) {

        String chaine ="test";

        test(chaine);

        System.out.println(chaine);

    }

}
```

Sortie du programme

```
test
```

Description du programme

System.out.println(chaine) affiche "test". Java passe les arguments par la valeurs, la méthode reçoit une copie de la référence. La modification test = test + test ne fait que réassigner la valeur locale test. La variable chaine dans main n'est pas modifiée car la modification ne concerne que la copie locale.

Question #5

Analyse préliminaire

Il faut déterminer quelle sera la valeur de la variable entier à la fin du code suivant et expliquer pourquoi. Il va falloir décortiquer la composition du code et définir nos variables. boolean a = false; boolean b = false;

int entier = (!a && (b | !a)) ? 10 : 20;

Description du programme

Décomposition de la condition (!a && (b | !a)) a = false

b = false

!a -> false = true

!a -> false = true

&& -> Et logique

b | !a -> false | true = true

!a && (b | !a) -> true && true = true

?: 10 : 20 -> Si true alors 10 sinon 20

Le résultat du code est 10, étant donnée que la condition (!a && (b | !a)) est vrai donc l'expression ternaire retourne la valeur de fauche, alors dans ce cas si 10.

Question 6

Analyse préliminaire

Il faut établir et expliquer en détail la différence entre les deux codes ci-dessous. L'explication doit inclure un exemple de code illusttrant la différence entre les deux morceaux de code.

Premier code :

public class Bonhomme {

public static String nom;

public Bonhomme(String n) {

```
nom = n;
```

}

}

Deuxième code :

public class Bonhomme {

public String nom;

public Bonhomme(String n) {

```
nom = n;
```

}

}

Au premier coup d'oeil, static n'est pas présent dans le deuxièeme, il va falloir définir à qui apartient la variable nom, soit la clase ou l'objet. Pour cela, nous allons exécuté les codes.

Code source

Code 1

```
public class Bonhomme {
  public static String nom;
  public Bonhomme(String n) {
    nom = n;
  }
    public static void main (String [] args) {
        Bonhomme b1 = new Bonhomme ("Rory");
        Bonhomme b2 = new Bonhomme ("Richard");

        System.out.println(b1.nom);
        System.out.println(b2.nom);

    }
}
```

Sortie du programme

```
Richard
Richard
```

Code 2

```
public class Bonhomme {
  public String nom;
  public Bonhomme(String n) {
    nom = n;
  }

    public static void main (String [] args) {
        Bonhomme b1 = new Bonhomme ("Rory");
        Bonhomme b2 = new Bonhomme ("Richard");

        System.out.println(b1.nom);
        System.out.println(b2.nom);

    }
}
```

Sortie du programme

```
Rory
Richard
```

Description du programme

Dans les deux codes la variable nom est de type String, cette variable a pour rôle de stocker le nom du bonhomme. Dans les deux code, b1 est le premier objet crée puis b2 est le second objet crée. La variable n de type String est le constructeur donc le paramètre reçu est d'initialiser la variable nom.

Le constructeur, Bonhomme (String n), reçoit un nom en paramètre puis assigne ce nom à la variable nom.

Le point d'entrée, main (String[] args), crée deux objets Bonhomme avec des noms différents, dans les exemples, les noms utilisés sont Rory et Richard, puis le point d'entrée affiche le nom de chaque objet avec System.out.println.

La différence entre le premier code contenant static et le second code qui est non-static est lors de la création des objets et en affichant leurs valeurs.

- Code 1 : nom est partagé -> b2 écrase b1, le résultat de b1 et b2 est Richard
- Code 2 : nom est indépendant -> b1 affiche Rory et b2 affiche Richard.

Ceci démontre concrètement qu'uen variable static appartient à la classe tandis qu'une variable non-static appartient à chaque objet.